



A IA Perfeita — Volume II: Prompts, Algoritmos e Skills da Cognição Determinística

O grimório técnico da conversa com a máquina — engenharia de prompts, algoritmos de raciocínio e a biblioteca viva de Skills Agênticas MMN_IA

Coletânea MMN_IA · A IA Perfeita · Vol. 2 de 3

Por Nexus HUB57 · Ecossistema MMN_IA

MMN_IA • 2026

Sobre este ebook

Se o Volume I tratou de como construir o **corpo** da IA Perfeita — sua arquitetura, seus padrões, seu código —, este Volume II trata da **mente**: o modo como conversamos com ela, os algoritmos que ela executa internamente, e as Skills (habilidades reaproveitáveis) que transformam um agente genérico em um especialista de alto valor. Aqui você encontra uma engenharia de prompts que vai muito além de "truques" — vai à teoria dos padrões CRISPE, PEAR, ReAct, Reflexion, Tree-of-Thought e Graph-of-Thought; algoritmos de roteamento, seleção e composição que sustentam a IA Perfeita; e a biblioteca viva de Skills Agênticas MMN_IA — habilidades como "Pesquisador Crítico", "Negociador Determinístico", "Auditor Documental", "Analista Quantitativo" e tantas outras. Dezenove capítulos, prompts de referência prontos para colar, e a doutrina cognitiva completa do ecossistema.

Sumário

- 1. Manifesto da cognição determinística • 2. Anatomia de um prompt MMN_IA • 3. Padrão CRISPE — Context · Role · Instruction · Steps · Personality · Expression • 4. Padrão PEAR — Persona · Expertise · Action · Result • 5. Padrão ReAct — Reason + Act, e por que ainda manda • 6. Padrão Reflexion — quando o agente revisa a si mesmo • 7. Padrão Tree-of-Thought (ToT) — explorar caminhos • 8. Padrão Graph-of-Thought (GoT) — raciocínio em rede • 9. Padrão Plan-and-Solve — separar planejamento de execução • 10. Algoritmos de roteamento: classificadores leves, embeddings, regras • 11. Algoritmos de seleção de exemplos (few-shot dinâmico) • 12. Algoritmos de auto-consistência (self-consistency, majority voting) • 13. O conceito de Skill no MMN_IA • 14. Estrutura de uma Skill — anatomia, contrato e versionamento • 15. A biblioteca MMN_IA de Skills (catálogo v1.0) • 16. Composição de Skills — meta-skills e pipelines compostos • 17. Eval de Skills — como medir habilidade reaproveitável • 18. Anti-padrões em prompts e Skills • 19. Checklist & fechamento — o grimório do operador

1. MANIFESTO DA COGNIÇÃO DETERMINÍSTICA

Existe um senso comum, vagamente romântico, segundo o qual conversar com uma IA é uma arte mágica — "saber pedir do jeito certo". Esse romantismo é o inimigo número um da IA Perfeita. **Conversar com IA, no padrão MMN_IA, é engenharia.** É escolha de padrão, é estrutura de prompt, é decisão sobre algoritmo de raciocínio, é versionamento, é teste, é métrica. A magia, quando aparece, é consequência da disciplina — não o contrário.

Cognição determinística é o regime em que **dado o mesmo input estrutural, o output é previsível em distribuição**. Não há "criatividade aleatória" no espaço operacional sério: há temperatura controlada, sampling controlado, exemplos controlados, schema controlado.

Este ebook é o **grimório técnico** dessa engenharia. Cada padrão aqui é uma fórmula que funciona em produção, escrita para Operadores que cansaram de improvisar e querem cultivar uma prática reaproveitável. A promessa: ao final, você terá um vocabulário de mais de quinze padrões cognitivos, uma biblioteca de Skills, e métodos para combinar tudo isso em sistemas previsíveis.

A diferença entre um Operador iniciante e um Operador sênior não é "saber mais prompts". É **saber escolher o padrão certo para o problema certo**.

2. ANATOMIA DE UM PROMPT MMN_IA

2.1 As sete partes de um prompt completo

- | | |
|-----------------|---|
| 1. CONTEXTO | – quem você é, onde opera, para quem trabalha |
| 2. OBJETIVO | – o que deve ser entregue, em uma frase |
| 3. ENTRADAS | – o que será fornecido (formato, exemplos) |
| 4. PROCEDIMENTO | – como pensar/agir, passo a passo |
| 5. REGRAS | – sempre / nunca / escalonamento |
| 6. FORMATO | – saída exata (JSON / markdown / outro) |
| 7. EXEMPLOS | – 1 a 5 pares input ↔ output |

2.2 Por que essa ordem importa

Modelos de fronteira em 2026 leem prompts inteiros, mas **pesam mais o início e o fim**. Colocar contexto e objetivo cedo + formato e exemplos no fim maximiza adesão.

2.3 Tamanho ideal

System prompt MMN_IA típico: **300-1.200 tokens**. Acima disso, retornos decrescem fortemente. Se está crescendo, é sinal para decompor em chamadas menores.

2.4 Versionamento

Todo system prompt mora em arquivo `.md` versionado em Git, com cabeçalho:

```
---
id: prompt_classifier_v2.3
owner: time_atendimento
last_updated: 2026-06-01
eval_score: 0.94
---
```

Sem isso, prompt é folclore — não engenharia.

3. PADRÃO CRISPE — CONTEXT · ROLE · INSTRUCTION · STEPS · PERSONALITY · EXPRESSION

3.1 O padrão

Acrônimo popularizado por Matt Nigh (2023), refinado no ecossistema MMN_IA:

- **Context** — situação, dado, restrição
- **Role** — papel que o modelo assume
- **Instruction** — tarefa específica
- **Steps** — procedimento passo-a-passo
- **Personality** — tom e estilo
- **Expression** — formato de saída

3.2 Template MMN_IA-CRISPE

```
[CONTEXT]
Trabalhamos com {dado/situação}. Restrição: {limite}.

[ROLE]
Você é {especialista X}, com {anos/qualificação} de experiência em {Y}.

[INSTRUCTION]
Sua tarefa: {ação clara}.

[STEPS]
1. {passo}
2. {passo}
3. {passo}

[PERSONALITY]
Tom: {profissional/técnico/empático/cordial}.

[EXPRESSION]
Devolva em {markdown / JSON / e-mail}.
```

3.3 Quando usar

Quando há **um papel claro** e **um procedimento conhecido**. Bom para: redação técnica, análise estruturada, atendimento padronizado, geração de relatórios.

3.4 Quando NÃO usar

Quando o problema é **exploratório** — para isso, ToT ou GoT. Quando exige uso de **ferramentas externas** — para isso, ReAct.

4. PADRÃO PEAR — PERSONA · EXPERTISE · ACTION · RESULT

4.1 O padrão

Padrão minimalista, ótimo para prompts curtos em produção de alta escala:

- **Persona** — quem você é
- **Expertise** — em que você é especialista
- **Action** — o que você faz
- **Result** — o que deve entregar

4.2 Exemplo

```
PERSONA: Analista financeiro sênior.  
EXPERTISE: Análise de balanços patrimoniais de PMEs.  
ACTION: Identificar três riscos contábeis no balanço anexo.  
RESULT: Lista priorizada (alto/médio/baixo) com justificativa em até 100 palavras cada.
```

4.3 Por que funciona

Resolve em 4 linhas o que CRISPE faz em 12. Para tarefas de baixa complexidade em alta escala, **PEAR ganha em custo**.

5. PADRÃO REACT — REASON + ACT, E POR QUE AINDA MANDA

5.1 A ideia

Cada turno do modelo é dividido em:

```
THOUGHT: ... (raciocínio interno)  
ACTION: ... (ferramenta + parâmetros)  
OBSERVATION: ... (resultado da ferramenta)  
THOUGHT: ... (próximo raciocínio)  
ACTION: ...  
...  
FINAL_ANSWER: ...
```

5.2 Por que continua sendo o coração agêntico

Modelos modernos têm function calling estruturado, mas internamente continuam executando ReAct. Entender ReAct é entender **o que um agente faz**.

5.3 Template MMN_IA-ReAct

Você é um agente do tipo ReAct. A cada turno produza:

THOUGHT: seu raciocínio

ACTION: nome_da_tool(parâmetros)

ou

FINAL_ANSWER: resposta ao usuário

Ferramentas disponíveis:

- search_web(query)
- read_doc(doc_id)
- run_sql(query)

Pare assim que tiver evidência suficiente.

5.4 Cuidados

- Limite **max_iterations** (5-10 típico)
- Limite **token budget**
- Trate "ação inválida" como erro de schema
- Registre todo o trace para replay

6. PADRÃO REFLEXION — QUANDO O AGENTE REVISA A SI MESMO

6.1 A ideia (Shinn et al., 2023)

Após produzir uma resposta, o agente:

1. Auto-avalia o resultado contra critérios
2. Identifica deficiências específicas
3. Reescreve com base no feedback gerado por si
4. Repete até atingir critério ou esgotar tentativas

6.2 Diferença para Critic Pattern

- **Critic**: outro agente avalia.
- **Reflexion**: o **próprio** agente avalia. Mais barato, menos imparcial.

A IA Perfeita usa **os dois juntos** quando a tarefa é crítica.

6.3 Template MMN_IA-Reflexion

```
[Após a resposta:]
Agora aja como crítico externo de sua própria resposta.
Identifique:
1. Pelo menos um ponto de incerteza não sinalizado
2. Pelo menos uma melhoria possível
3. Se há afirmação não suportada

Em seguida, produza uma versão revisada.
```

6.4 Quando paga o custo

Tarefas de redação, análise, raciocínio. Ganho médio observado: +10-25% em qualidade humana percebida.

7. PADRÃO TREE-OF-THOUGHT (ToT) — EXPLORAR CAMINHOS

7.1 A ideia (Yao et al., 2023)

Em vez de raciocinar linearmente, o agente **explora múltiplos caminhos** de raciocínio em paralelo:

```
      [problema]
      /   |   \
    ramo A ramo B ramo C
    / \   / \   / \
  ...  ...  ...  ...  ...
      ↓ avalia ramos ↓
    ramo vencedor
```

7.2 Quando usar

Problemas com **múltiplas estratégias possíveis** e onde a estratégia certa não é óbvia: planejamento, design, escolha estratégica, problemas algorítmicos.

7.3 Custo

ToT pode multiplicar custo de tokens por 3-10x. Use com critério.

7.4 Template MMN_IA-ToT

1. Gere 3 abordagens distintas para o problema.
2. Para cada uma, descreva o procedimento em 4-6 passos.
3. Avalie cada abordagem em: viabilidade (1-5), custo (1-5), risco (1-5).
4. Recomende a melhor com justificativa em 80 palavras.

8. PADRÃO GRAPH-OF-THOUGHT (GoT) — RACIOCÍNIO EM REDE

8.1 A ideia (Besta et al., 2023)

Generaliza ToT permitindo que **caminhos se cruzem, mesquem e dividam**, formando um grafo:

```
[problema]
  ↓
[sub-problema A] ↔ [sub-problema B]
  ↓             ↓
[solução parcial A] [solução parcial B]
  ↘             ↙
      [síntese]
```

8.2 Quando vale a complexidade

Sistemas que precisam de **decomposição de problemas complexos** com **reuso de sub-soluções**: planejamento multi-agente, refatoração de código grande, análise de portfólio.

8.3 Implementação prática

Em 2026, GoT é mais frequentemente implementado como **orquestração explícita em código**, não como prompt único. O LLM resolve cada nó; o grafo é gerenciado externamente.

9. PADRÃO PLAN-AND-SOLVE — SEPARAR PLANEJAMENTO DE EXECUÇÃO

9.1 A ideia

Em vez de pedir solução direta, peça primeiro o **plano**:

```
TURNO 1: produza um plano em até N passos.
TURNO 2: execute o plano, passo por passo, mostrando resultado de cada um.
```

9.2 Por que funciona

Modelos cometem menos erros quando **planejam antes de agir**. Inspirado em humanos, validado em benchmarks.

9.3 Template

```
[FASE 1 - PLAN]
Para o problema: {x}
Liste 4-7 passos lógicos, sem executar nenhum.

[FASE 2 - SOLVE]
Execute o plano que você gerou. Em cada passo, mostre:
- O que pretende fazer
- Como faz
- Qual o resultado
- Validação interna
```

10. ALGORITMOS DE ROTEAMENTO: CLASSIFICADORES LEVES, EMBEDDINGS, REGRAS

10.1 Por que roteamento importa

Para reduzir custo e aumentar qualidade, **roteie cada query para o agente/modelo certo**. Não chame Claude Opus para classificar uma pergunta simples — use Haiku.

10.2 Três técnicas

Técnica	Custo	Precisão	Quando usar
Regex / keyword	Quase zero	Baixa-Média	Casos óbvios
Embedding + KNN	Baixo	Alta	Domínios estáveis
LLM-classifier leve	Baixo-Médio	Muito alta	Ambíguo / dinâmico

10.3 Padrão híbrido MMN_IA

```
1. Regex catch (90% dos casos óbvios)
2. Embedding KNN (5% intermediários)
3. LLM-classifier leve (5% ambíguos)
```

Resultado: 95% das queries roteadas a custo trivial; 5% pagam por precisão.

11. ALGORITMOS DE SELEÇÃO DE EXEMPLOS (FEW-SHOT DINÂMICO)

11.1 O problema

Exemplos few-shot são poderosos, mas **incluir muitos** consome tokens e diminui qualidade. Solução: selecionar dinamicamente os exemplos **mais relevantes** para cada query.

11.2 Receita

1. Indexar exemplos rotulados em vector DB
2. Para cada query nova, embed e buscar top-3 similares
3. Incluir esses 3 no prompt como few-shot
4. Eventualmente reranker para refinar

11.3 Ganho típico

Few-shot dinâmico vs fixo: **+15-30% em accuracy** em tarefas de classificação/extração, com **mesma janela de contexto**.

12. ALGORITMOS DE AUTO-CONSISTÊNCIA (SELF-CONSISTENCY, MAJORITY VOTING)

12.1 A ideia (Wang et al., 2022)

Em vez de uma única amostra com temperatura 0, gere **N amostras** (temperatura > 0) e tome o **resultado majoritário**.

12.2 Quando funciona

Tarefas com **resposta discreta e verificável**: classificação, matemática, raciocínio simbólico.

12.3 Custo

Multiplica custo por N (típico N=3-10). Compensa em domínios onde erro é caro.

12.4 Template

```
Para a pergunta {x}, gere 5 respostas independentes, cada uma
segundo raciocínio próprio. Devolva-as como lista.

[chamada paralela]

Out: list[Answer]
→ majority_vote(list[Answer]) # determinístico, em código
```

13. O CONCEITO DE SKILL NO MMN_IA

13.1 Definição

Uma **Skill** é uma **habilidade reaproveitável** empacotada como um artefato versionado contendo: prompt(s), schemas, exemplos, eval suite e metadados. Skills são para LLMs o que **funções nomeadas** são para programação.

13.2 Por que Skills mudam o jogo

Sem Skills, cada projeto reinventa redação, análise, classificação. Com Skills, você **compõe** sistemas a partir de blocos testados. Time produtivo MMN_IA não escreve prompt do zero — ele instancia Skill da biblioteca.

13.3 Skill vs Agente vs Prompt

Nível	O que é	Granularidade
Prompt	Texto único enviado ao LLM	Micro
Skill	Capacidade reaproveitável (prompt + schema + eval)	Meso
Agente	Sistema que combina Skills + tools + memória	Macro

14. ESTRUTURA DE UMA SKILL — ANATOMIA, CONTRATO E VERSIONAMENTO

14.1 Os arquivos de uma Skill

```
skills/
├─ pesquisador_critico/
│  ├─ README.md           # descrição, casos de uso
│  ├─ skill.yaml          # metadata
│  ├─ system_prompt.md    # prompt principal
│  ├─ input_schema.json   # JSON Schema da entrada
│  ├─ output_schema.json  # JSON Schema da saída
│  ├─ examples/           # few-shot fixos
│  │  ├─ ex_01.json
│  │  └─ ex_02.json
│  ├─ evals/              # casos de teste
│  │  ├─ eval_001.json
│  │  └─ ...
│  └─ CHANGELOG.md
└─
```

14.2 O arquivo `skill.yaml`

```
id: pesquisador_critico
version: 1.4.0
owner: mmn_ia.knowledge
model_default: claude-4.5-sonnet
model_fallback: gpt-5-mini
temperature: 0.2
max_tokens: 4096
description: |
  Skill que investiga uma afirmação, identifica evidências
  pró e contra, e devolve um relatório estruturado com
  faithfulness alta.
tags: [research, critical_thinking, source_verification]
eval_score: 0.91
```

14.3 O contrato

Toda Skill tem **contrato** explícito:

- O que ela aceita (input schema)
- O que ela devolve (output schema)
- Quando deve recusar (refusal cases)
- Quando deve escalar (escalation cases)
- Qual a métrica de qualidade (eval score)

Skill sem contrato é prompt fingindo ser Skill.

15. A BIBLIOTECA MMN_IA DE SKILLS (CATÁLOGO v1.0)

15.1 Catálogo de Skills disponíveis

Skill	Domínio	Função
pesquisador_critico	Knowledge	Pesquisa com verificação de fontes
negociador_deterministico	Vendas/Compras	Negociação com regras explícitas
auditor_documental	Compliance	Auditoria de documentos
analista_quantitativo	Dados	Análise estatística + interpretação
redator_executivo	Comunicação	Texto executivo curto, alto impacto
revisor_juridico_basico	Direito	Revisão de contratos padrão
classificador_intencao	Atendimento	Triagem de mensagens
extrator_estruturado	Dados	Extração schema-driven
traducao_tecnica	Localização	Tradução com terminologia técnica
gerador_resumo_executivo	Knowledge	Resumo TL;DR de longos docs
planejador_tatico	Operações	Plan-and-Solve de projetos
explicador_didatico	Educação	Explicação por nível do leitor
gerador_caso_uso	Produto	Geração de casos de uso a partir de problema
avaliador_qualidade	QA	LLM-as-judge calibrado
decisor_zona_amarela	Operações	Decisão em zonas de risco médio
negociador_empatico	Atendimento	Resolução de conflito
coletor_requisitos	Engenharia	Levantamento de requisitos
codificador_revisor	DevOps	Code review automatizado
sintetizador_pesquisa	Research	Síntese de múltiplas fontes
executor_workflow	Operações	Execução passo-a-passo de SOPs

15.2 Princípio de evolução do catálogo

- Skills são **adicionadas** por contribuição com PR + eval mínimo (0.85)
- Skills são **deprecadas** com aviso de 90 dias e migração documentada
- Skills são **versionadas** semver — major/minor/patch com changelog

15.3 Exemplo — Skill `pesquisador_critico`

System prompt resumido (v1.4):

```
Você é o Pesquisador Crítico do ecossistema MMN_IA.
Sua função: dada uma afirmação ou pergunta, investigar,
identificar evidências pró e contra, classificar
confiança e devolver um relatório estruturado.

PROCEDIMENTO:
1. Quebre a afirmação em sub-afirmações verificáveis.
2. Para cada uma, busque (via tool search) ≥2 fontes.
3. Avalie cada fonte: tipo, data, confiabilidade.
4. Classifique cada sub-afirmação:
   SUPPORTED | CONTESTED | REFUTED | INSUFFICIENT_EVIDENCE.
5. Sintetize com confiança calibrada.

REGRAS:
- Nunca afirme algo sem fonte.
- Sempre marque incerteza.
- Recuse opinar sobre conteúdo especulativo.

SAÍDA: JSON conforme output_schema.json.
```

16. COMPOSIÇÃO DE SKILLS — META-SKILLS E PIPELINES COMPOSTOS

16.1 Skills se combinam

Uma **meta-skill** é uma Skill que compõe outras Skills em pipeline. Exemplo: skill `gerador_relatorio_executivo` chama, em sequência:

```
[ coletor_requisitos ]
    ↓
[ pesquisador_critico ]
    ↓
[ analista_quantitativo ]
    ↓
[ sintetizador_pesquisa ]
    ↓
[ redator_executivo ]
    ↓
[ avaliador_qualidade ]
```

16.2 Vantagem da composição

- Cada bloco é testado isoladamente
- Cada bloco evolui independentemente
- O pipeline tem **observabilidade granular**
- Falha de um bloco não corrompe o resto se o handoff for limpo

16.3 Anti-padrão de composição

Tentar resolver pipeline complexo em **um único super-prompt**. Repete o erro do "Frankenstein-prompt". Compose.

17. EVAL DE SKILLS — COMO MEDIR HABILIDADE REAPROVEITÁVEL

17.1 As quatro dimensões de eval de Skill

Dimensão	O que mede
Correctness	A saída está certa?
Faithfulness	Está ancorada em fonte/input?
Refusal accuracy	Recusa o que deve? Aceita o que deve?
Stylistic adherence	Mantém tom e formato?

17.2 Suite eval mínima por Skill

- ≥ 50 casos rotulados
- ≥ 5 casos adversariais
- ≥ 3 casos de borda

- ≥ 3 casos de "deve recusar"

17.3 Score mínimo de produção

```
Correctness ≥ 0.85
Faithfulness ≥ 0.90 (se aplicável)
Refusal ≥ 0.95
Style ≥ 0.85
```

Abaixo disso, Skill fica em "preview", não em "stable".

17.4 Eval comparativa em CI

A cada mudança de prompt/modelo, suite roda em CI. Resultado vai para tabela comparativa. Promoção exige **score igual ou superior** em todas as dimensões.

18. ANTI-PADRÕES EM PROMPTS E SKILLS

Anti-padrão	Sintoma	Antídoto
Polidez performática ("por favor, faça com excelência")	Tokens desperdiçados	Imperativo claro
"Pense passo-a-passo" copy-paste	Aplicado fora de contexto	Use ReAct/CoT só onde ajuda
Few-shot único genérico	Não generaliza	Few-shot dinâmico
Saída em texto livre	Parsing frágil	Schema-driven
Sem versão	Folclore, não engenharia	Git + semver
Sem eval	Mudanças cegas	Eval suite obrigatória
Prompt-anonimato ("aja como um expert")	Vago demais	PEAR com persona específica
Modelo escolhido por hábito	Caro ou inadequado	Routing por tarefa
Sem critério de parada agêntica	Loops e custo	Limites duros
Dependência de quirks de um modelo	Quebra na próxima versão	Padrão portátil

19. CHECKLIST & FECHAMENTO — O GRIMÓRIO DO OPERADOR

19.1 Checklist do prompt de produção

- ☐ Anatomia 7-partes presente (ou justificada)
- ☐ Padrão escolhido (CRISPE/PEAR/ReAct/...) declarado
- ☐ Saída em schema validado
- ☐ Few-shot dinâmico ou justificado fixo
- ☐ Versionado em Git com semver
- ☐ Eval suite ≥ 50 casos
- ☐ Score de produção em todas as dimensões
- ☐ Documentação de quando NÃO usar
- ☐ Dono nomeado e contato
- ☐ Plano de evolução

19.2 Checklist da Skill MMN_IA-compliant

- ☐ Pasta skill com README, skill.yaml, prompts, schemas, examples, evals
- ☐ skill.yaml com metadata completo
- ☐ Contrato input/output explícito
- ☐ Refusal cases listados
- ☐ Escalation cases listados
- ☐ Eval ≥ 50 casos com score por dimensão
- ☐ Changelog atualizado
- ☐ Composição: Skill aparece em pelo menos 1 meta-skill OU é raiz
- ☐ Owner ativo no ecossistema

19.3 Fechamento

A cognição determinística não tem fim — é uma prática contínua. Cada novo modelo abre padrões novos; cada caso de uso revela uma Skill nova; cada eval expõe um anti-padrão antigo. O Operador que dominar esse grimório tem uma vantagem composta: enquanto outros reaprendem do zero a cada modelo, ele **migra suas Skills**, evolui sua biblioteca, e mantém um nível de qualidade que parece sobre-humano para quem está olhando de fora.

"What's in a name? That which we call a rose / By any other name would smell as sweet." — Shakespeare, *Romeu e Julieta*, Ato II, Cena II

Mas em engenharia de IA, o nome **importa**. Um padrão chamado errado é um padrão sem dono, sem versão, sem eval. Nomeie suas Skills. Versione seus prompts. Construa seu grimório.

Continue a Coletânea

Vol. 3 — *A IA Perfeita · Autocura, Autoconhecimento e Sabedoria Agêntica Determinística Sistêmica*
fecha a trilogia explorando como Skills se tornam **maturidade**: sistemas que se monitoram, se reparam, se evoluem e atingem o estado de **sabedoria operacional**.

Acesse: github.com/Nexus-HUB57/MMN_AI-to-AI · [docs/ebooks_markdown/colecao_A_IA_Perfeita/](#)

Nexus HUB57 · Ecossistema MMN_IA · 2026